

Шпаргалка по CI/CD

Понятия непрерывной интеграции и поставки, устройство конвейера, примеры для GitHub Actions и GitLab CI, работа с секретами и артефактами, стратегии выкладки и признаки здорового процесса.

Слова и различия

- **CI (непрерывная интеграция)** — каждое изменение попадает в общую ветку и автоматически собирается и тестируется.
- **Continuous Delivery** — каждая проверенная сборка готова к выкладке, решение о ней принимает человек.
- **Continuous Deployment** — выкладка происходит сама, без ручного шага.
- **Конвейер** — последовательность этапов; падение раннего этапа останавливает остальные.
- **Раннер (агент)** — машина или контейнер, где выполняются шаги.
- **Артефакт** — результат сборки, который передаётся дальше по конвейеру или хранится.

Типовые этапы

1. Проверка кода
Линтеры, форматирование, статический анализ — самые быстрые проверки идут первыми
2. Сборка
Компиляция, упаковка образа, сохранение артефакта
3. Тесты
Модульные, затем интеграционные; параллелить по частям, если долго
4. Безопасность
Проверка зависимостей и образов, поиск секретов в коде
5. Выкладка на тестовый контур
Автоматически, с прогоном проверок после развёртывания
6. Выкладка в рабочую среду
По подтверждению или автоматически, с возможностью быстрого отката

GitHub Actions

```
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: npm
      - run: npm ci
      - run: npm test
      - uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage/
```

strategy.matrix

Один и тот же набор шагов на нескольких версиях или платформах

needs: [build]

Задание ждёт завершения указанного

if: github.ref == 'refs/heads/main'

Условное выполнение шага или задания

secrets.TOKEN, vars.REGION

Секреты и обычные переменные репозитория

environment: production

Среда с правилами защиты: ручное подтверждение, ограничение веток

concurrency

Не выполнять две сборки одной ветки одновременно

GitLab CI

```
stages: [build, test, deploy]

variables:
  IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA

build:
  stage: build
  script:
    - docker build -t $IMAGE .
    - docker push $IMAGE

test:
  stage: test
  script: [make test]
  artifacts:
    reports:
      junit: report.xml

deploy:
  stage: deploy
  script: [./deploy.sh]
  environment: production
  when: manual
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Полезные переменные: CI_COMMIT_SHA, CI_COMMIT_BRANCH, CI_PIPELINE_ID, CI_PROJECT_DIR.

Кеш и артефакты

- Кеш**
Ускоряет сборку (зависимости, промежуточные файлы). Может пропасть — конвейер должен работать и без него
- Артефакт**
Результат, который нужен дальше или людям: пакеты, образы, отчёты
- Ключ кеша**
Обычно хеш файла зависимостей: package-lock.json, go.sum, requirements.txt
- Срок хранения**
Ограничивайте: артефакты быстро съедают место и деньги

Секреты

- Хранить только в секретах платформы или во внешнем хранилище (Vault, облачный менеджер секретов) — никогда в репозитории.
- Выдавать по принципу наименьших прав и на конкретную среду.
- Предпочитать короткоживущие маркеры и OIDC-доступ к облаку вместо постоянных ключей.
- Для форков секреты по умолчанию недоступны — это защита, а не ошибка настройки.
- Регулярно менять ключи и следить, чтобы они не попадали в журналы сборки.

Версии и релизы

- Семантическое versionирование**
МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ: несовместимые изменения, новые возможности, исправления
- Теги как источник релиза**
Сборка по тегу даёт воспроизводимый артефакт с понятным именем
- Неизменяемые артефакты**
Один и тот же образ проходит все среды; меняются только настройки
- Файл изменений**
Формируется из сообщений коммитов, если принят единый формат (например, Conventional Commits)

Стратегии выкладки

- Поочерёдное обновление**
Экземпляры заменяются по одному; простая и распространённая схема
- Сине-зелёная**
Две одинаковые среды, переключение трафика целиком, мгновенный откат
- Канареечная**
Небольшая доля трафика на новую версию, наблюдение за метриками, затем расширение
- Флаги функций**
Код выложен, но возможность включается отдельно — выкладка отделена от запуска

Признаки здорового конвейера

- Основная проверка на pull request укладывается в 10–15 минут, иначе её начинают обходить.
- Красный результат означает реальную проблему: «мигающие» тесты чинят, а не перезапускают.
- Сборка воспроизводима: зафиксированные версии зависимостей и образов.
- Одинаковые шаги можно выполнить локально — те же команды, что и в конвейере.
- Откат отработан и занимает минуты, а не часы.
- Права раннеров ограничены: доступ в рабочую среду только у этапа выкладки.

Метрики

- Частота выкладок** — как часто изменения доходят до пользователей.
- Время от коммита до рабочей среды** — длина пути изменения.
- Доля неудачных выкладок** — сколько релизов потребовали исправления.
- Время восстановления** — сколько занимает возврат к рабочему состоянию.